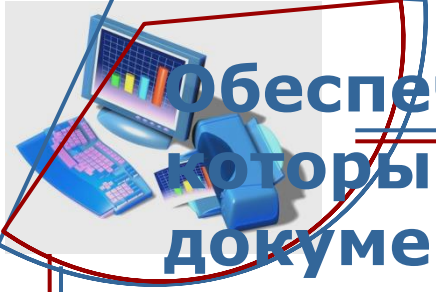


Обеспечение качества - это процесс, который включает в себя установление и документирование требований к продукции с учетом характеристик качества, а также контроль и обеспечение соответствия продукции этим требованиям

1. **Модель качества ПО**
2. Требования к качеству ПО
3. Измерения качества ПО
4. Оценка качества ПО

Надёжность –это способность ПС безотказно выполнять определенные функции при заданных условиях в течение заданного периода времени.

Надёжность определяется:

- завершенность
- устойчивость к отказам
- способность к восстановлению
- согласованность

Насколько оно должно быть надежно, например:

- *работать 7 дней в неделю и 24 часа в сутки;*
- *допускается неработоспособность в течение не более 3 часов в год;*
- *никакие введенные пользователями данные при отказе не должны теряться.*

Надежность	
Завершенность, зрелость (completeness)	степень обладания ПС всеми необходимыми частями и чертами, требующимися для выполнения своих явных и неявных функций
Устойчивость к отказам, отказоустойчивость (robustness fault tolerance)	способность ПС продолжать корректное функционирование, несмотря на возникновение отказа
Способность к восстановлению (recoverability) –	способность восстанавливать определенный уровень работоспособности и целостность данных после отказа, необходимые для этого время и ресурсы.
Соответствие стандартам надежности (reliability compliance)	

Устойчивость: отказоустойчивость программного средства или обеспечение толерантности

- безотказное функционирование при отказах,
- минимизация возможных потерь данных
- исключение опасных действий при внезапном нарушении условий функционирования

Принципы надежности:

- <u>Предупреждение ошибок</u>
- Обнаружение
- Исправление
- Обеспечение устойчивости к ошибками

- *Борьба со сложностью*
- ***обеспечение точности перевода***
- *преодоление барьера между пользователем и разработчиком*

Коммуникационная модель

обеспечение точности перевода

Коммуникационная модель

Борьба со сложностью

- Модульная архитектура
- SOLID
- Ограничение состояний системы

Иммутабельные (неизменяемые) объекты

```
public final class User {
    private final String name;
    private final int age;

    public User(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Только геттеры, нет сеттеров!
    public String getName() { return name; }
    public int getAge() { return age; }

    // "Изменение" через создание нового
объекта
    public User withAge(int newAge) {
        return new User(this.name, newAge);
    }
}
```

Конечные автоматы (Finite State Machines, FSM)

```
public class Order {
    private enum State { CREATED, PAID, DELIVERED, CANCELLED }

    private State currentState;

    public Order() {
        this.currentState = State.CREATED;
    }

    public void pay() {
        if (currentState != State.CREATED) {
            throw new IllegalStateException("Нельзя оплатить заказ в состоянии " + currentState);
        }
        currentState = State.PAID;
    }

    public void deliver() {
        if (currentState != State.PAID) {
            throw new IllegalStateException("Нельзя доставить неоплаченный заказ");
        }
        currentState = State.DELIVERED;
    }

    // Другие методы...
}
```

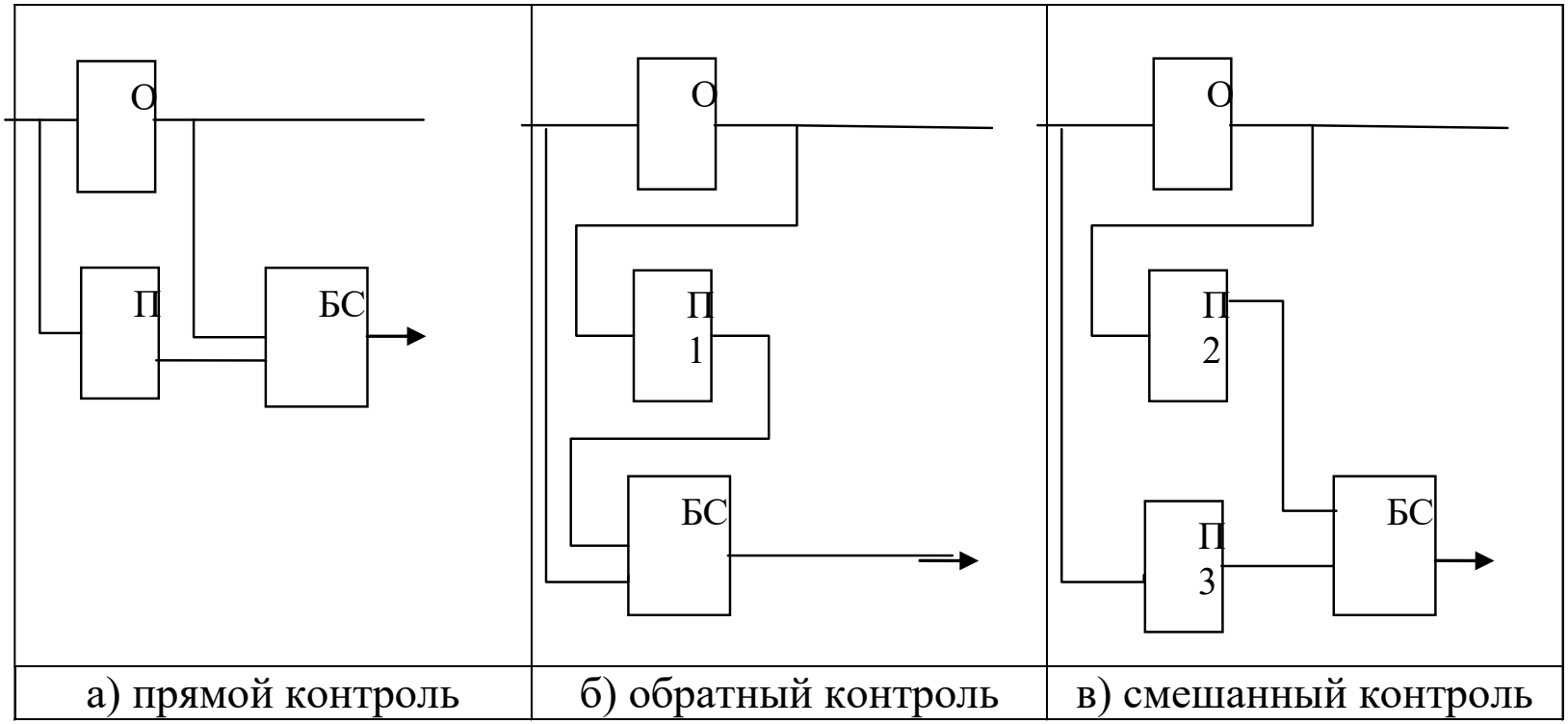
Event Sourcing

Пример: Банковский счёт:

События: [Открытие(100₽), Перевод(+50₽), Списание(-30₽)]

Текущее состояние: 120₽.

№2 - Обнаружение ошибок в коде программе



```
public double divide(int a, int b) {
    if (b == 0) {
        throw new IllegalArgumentException("Делитель
не может быть нулём");
    }
    return a / (double) b;
}
```

```
public String loadDataFromFile(String path) {
    String data = readFile(path); // Может вернуть null
    if (data == null) {
        throw new RuntimeException("Файл не удалось прочитать");
    }
    return data;
}
```

Утверждение (assert) — это оператор (statement) языка позволяет вам проверить свои предположения о программе

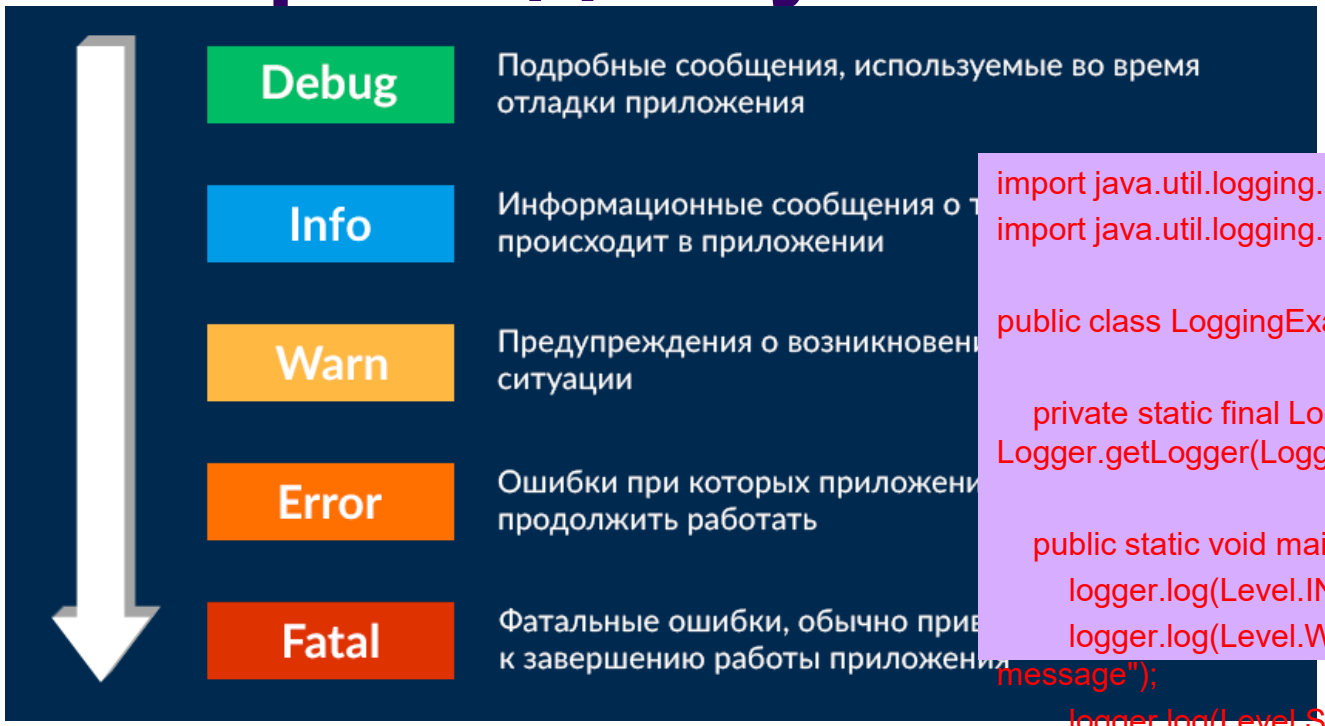
```
switch(suit) {
   caseSuit.CLUBS:        // бубна
      ...
      break;
   caseSuit.DIAMONDS:  // трефа
      ...
      break;
   caseSuit.HEARTS:      // черва
      ...
      break;
   caseSuit.SPADES:      // пика
      ...
}
```

```
default:
    assert false : suit;
```

```
default:
    throw new AssertionError(suit);
```

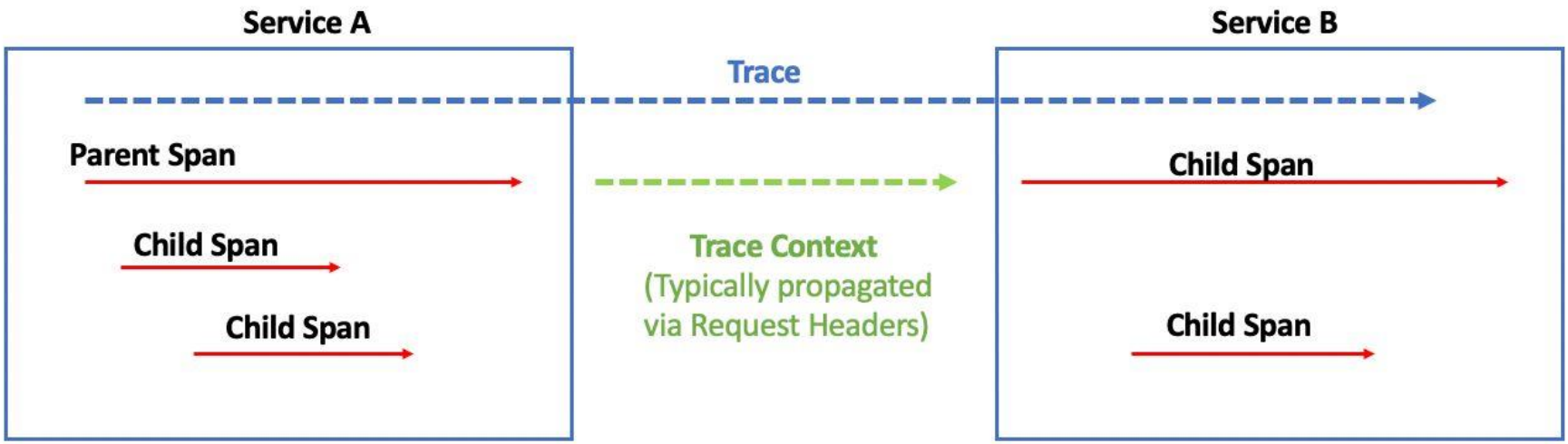
Логгирование - фиксация и структурирование информации о работе системы в отдельные лог-файлы с возможностью быстрого доступа

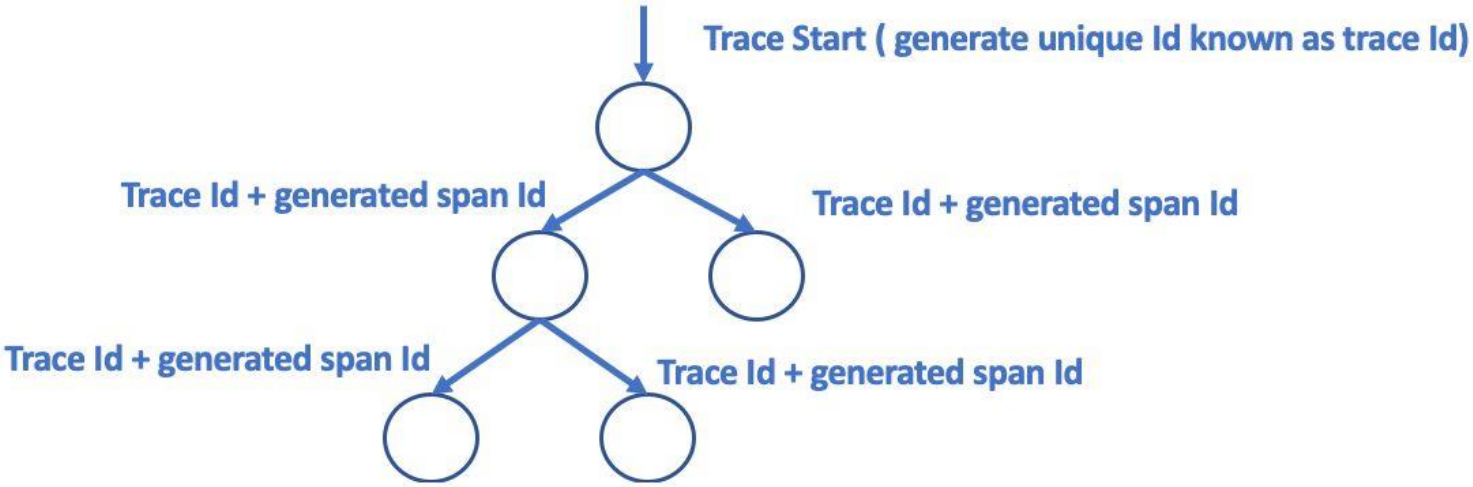
- Log4j или Logback




```
import java.util.logging.Level;
import java.util.logging.Logger;

public class LoggingExample {

    private static final Logger logger =
Logger.getLogger(LoggingExample.class.getName());

    public static void main(String[] args) {
        logger.log(Level.INFO, "This is an info message");
        logger.log(Level.WARNING, "This is a warning
message");
        logger.log(Level.SEVERE, "This is a severe
message");
    }
}
```

трассировка программы

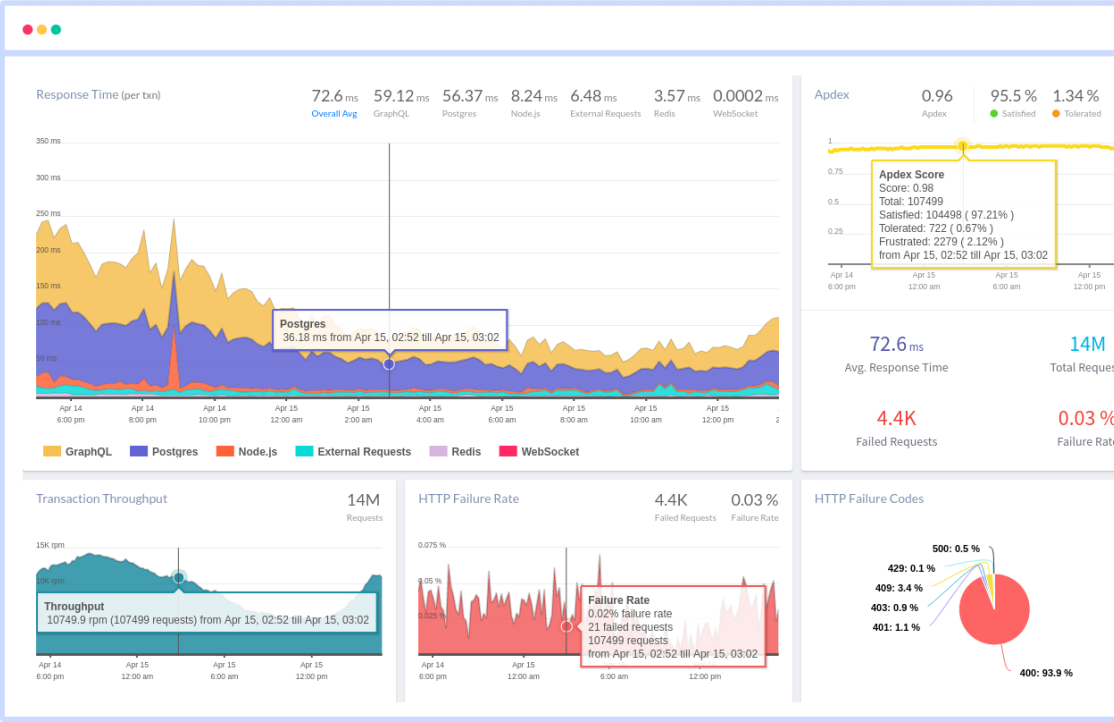




```
String traceId = UUID.randomUUID().toString();
    MDC.put("traceId", traceId);

    logger.info("Начало выполнения программы");

    int inputValue = new Random().nextInt(100);
    logger.info("Получено входное значение: {}", inputValue);

    processValue(inputValue);

    logger.info("Завершение выполнения программы");
    MDC.remove("traceId");
```

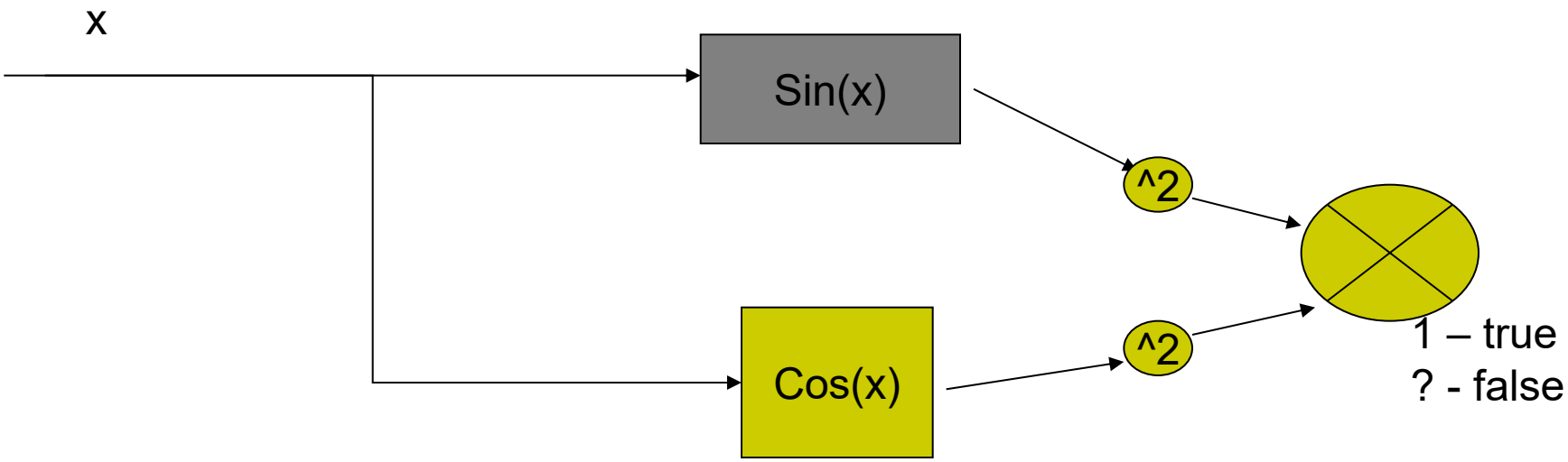
Метрики приложения

- Показатели использования ресурсов, таких как процессор, память, дисковое пространство, пропускная способность сети и т.д.
- Показатели производительности такую как время отклика, задержка, пропускная способность
- Показатели частоты ошибок:, исключений или сбоев, возникающих в системе.
- Показатели насыщенности ресурса, например, количество активных подключений, открытых файлов и т.д.
- Показатели доступности: например, процент времени, в



№3 - исправление ошибок в программе

Метод контрольных функций



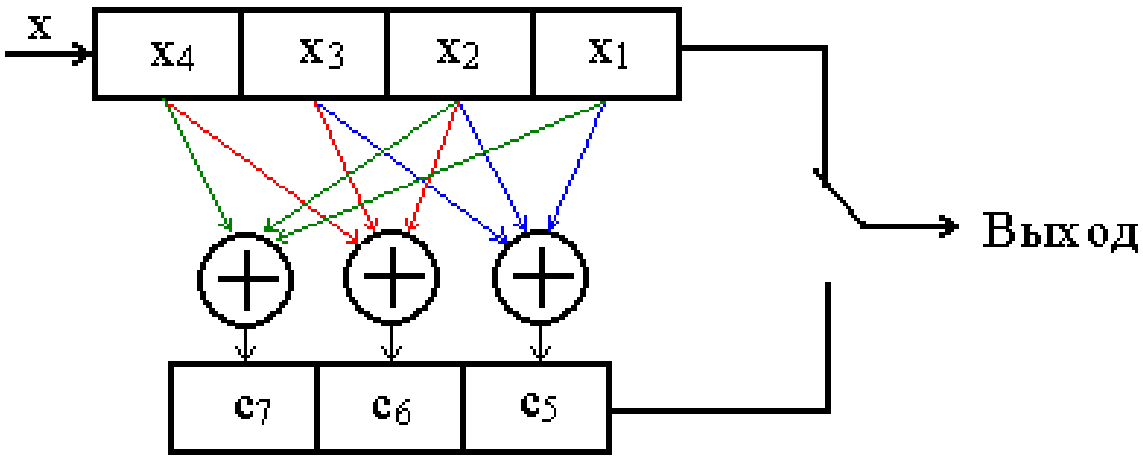
Контроль целостности при передаче
При отправке данных вычисляется хеш
Получатель проверяет хеш полученных данных
Если хеши не совпадают → данные повреждены

Обнаружение изменений в хранилищах

Верификация загружаемых файлов

```
// Проверка скачанного файла String expectedHash = "d0a3c..."; // Хеш из доверенного источника
byte[] fileContent = Files.readAllBytes(Paths.get("update.jar"));
 if(!verifyData(fileContent, expectedHash)) { throw new SecurityException("Файл повреждён или изменён!"); }
```

Избыточное кодирование

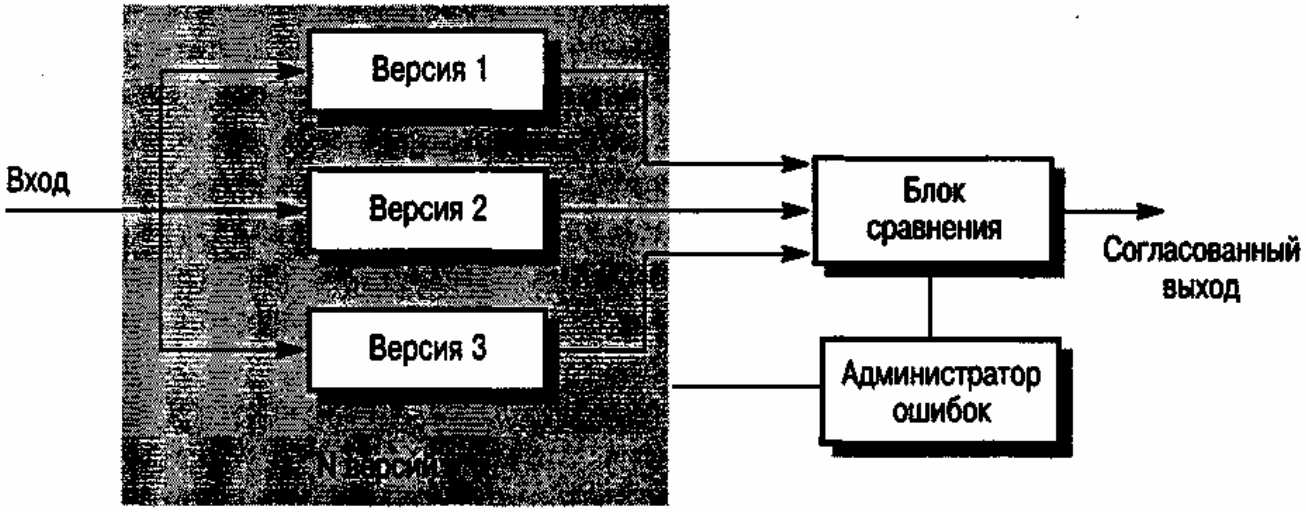


№4 Обеспечение устойчивости программы к ошибкам

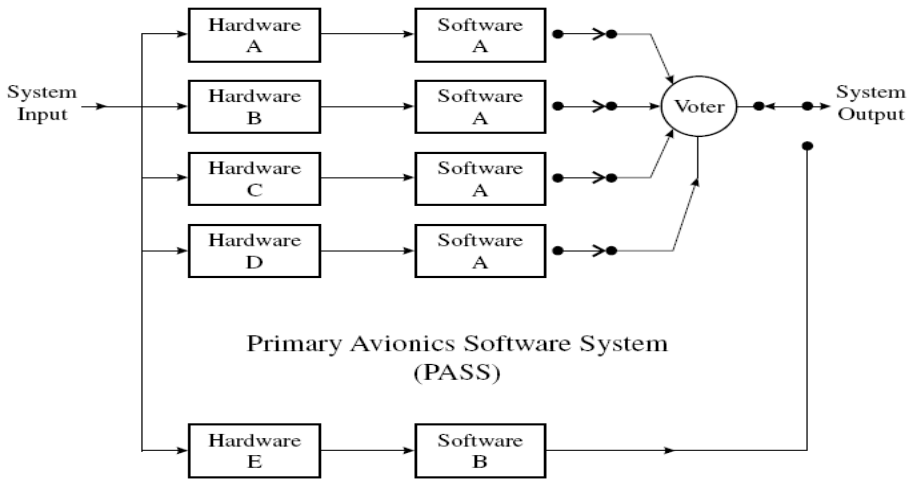
- ? Технические подходы

- динамическая избыточность
- методы отступления
- методы изоляции ошибок

N-вариантное программирование



Пример: Система управления Шаттлом

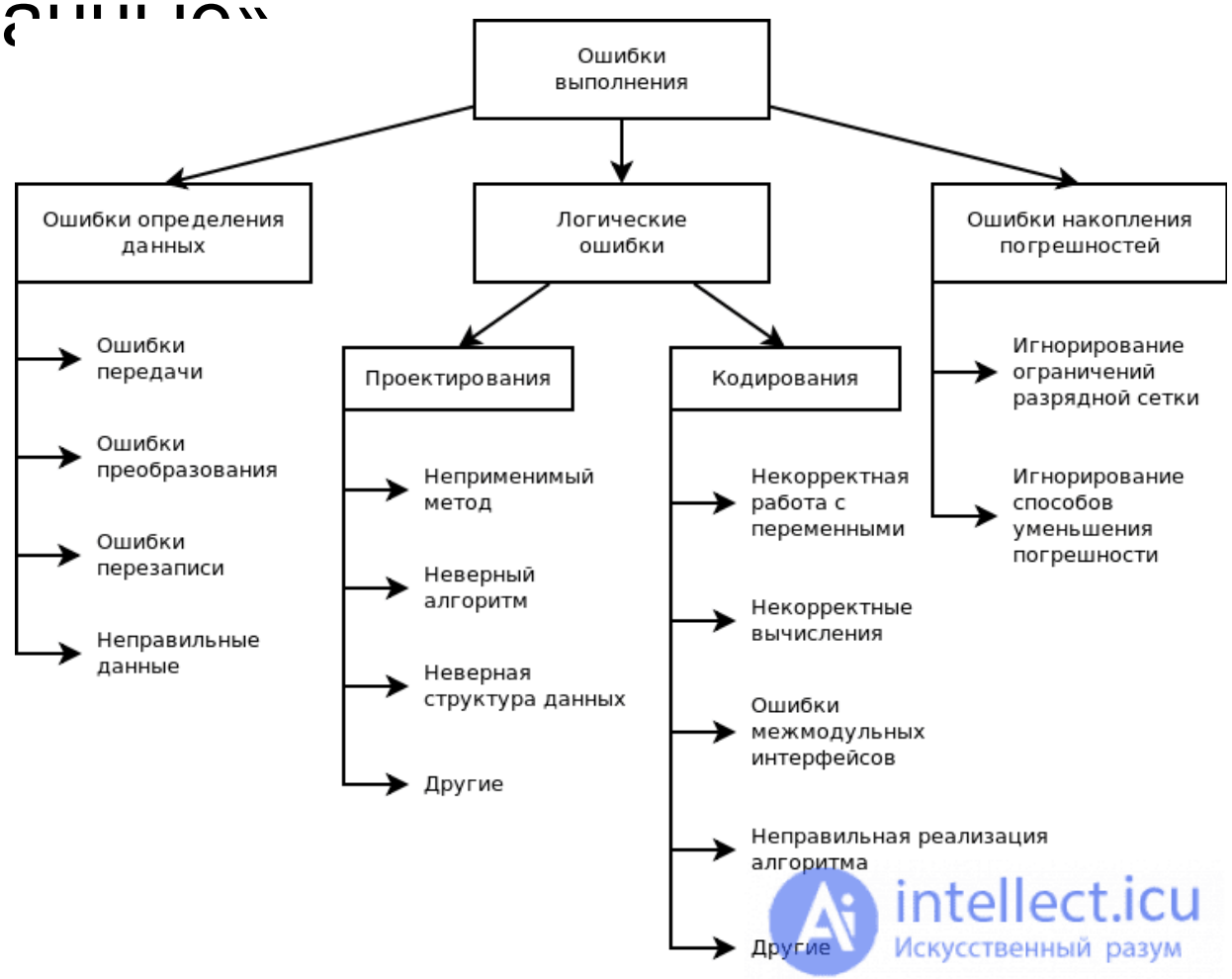


- Для нормального полета "Шаттла" необходим один компьютер. Чтобы обезопасить себя от возможных неисправностей, учитывая возможность ошибок программирования, пять одинаковых компьютеров выполняют две разные программы. Две пары компьютеров, работающие с программным обеспечением основного электронного оборудования, проверяют расчеты друг друга 440 раз в секунду и отбрасывают те результаты, которые наиболее сильно отличаются от трех других. Пятый компьютер, который выполняет резервную программу полета, может взять управление на себя при определенных условиях; это же может быть сделано экипажем. На наиболее ответственных этапах полета - выведении на орбиту и спуске - работают все пять компьютеров. Для рутинных операций на орбите достаточно одного или двух компьютеров, а остальные находятся в горячем резерве или выключены.

V-спецификаций

Восстанавливаемость

- «нет безошибочных программ, есть недотестированные»

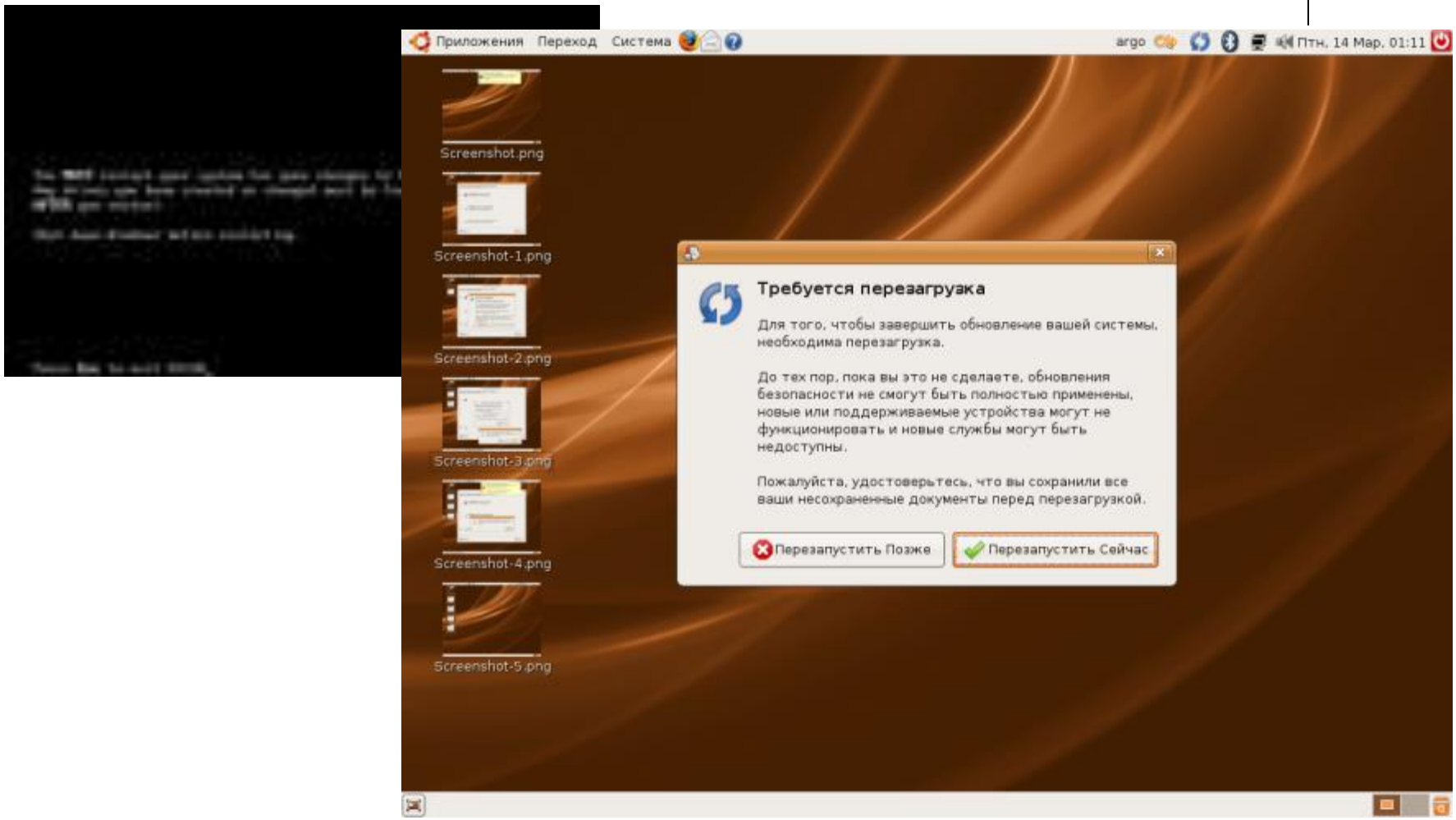


intellect.icu
Искусственный разум

Методы отступления

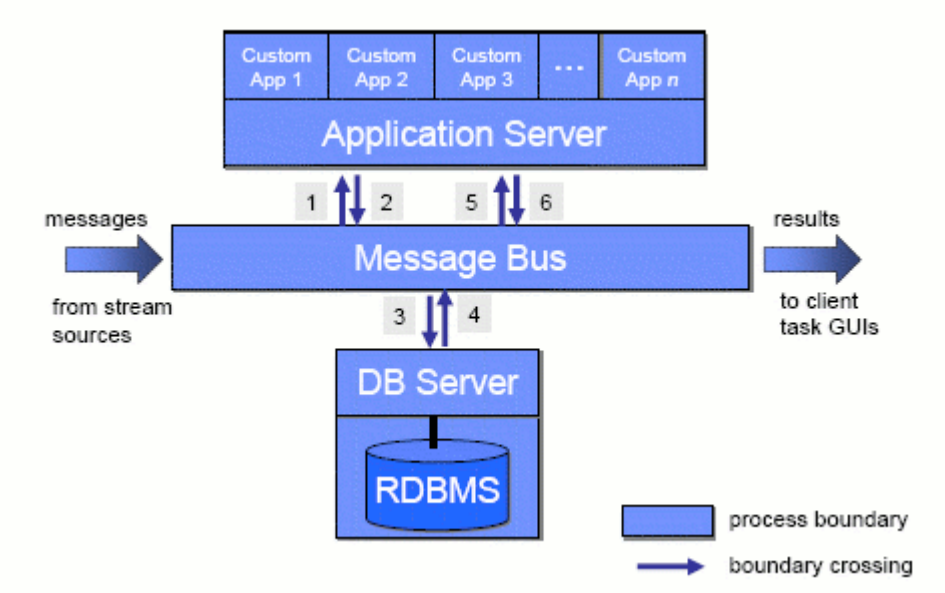
- методы перезагрузки (рестарта);
- журналирование;
- методы отката;
- контрольные точки.

Reboot


Приложения
Переход
Система
argo
Птн, 14 Мар, 01:11
Screenshot.png
Screenshot-1.png
Screenshot-2.png
Screenshot-3.png
Screenshot-4.png
Screenshot-5.png
Требуется перезагрузка
Для того, чтобы завершить обновление вашей системы, необходима перезагрузка.
До тех пор, пока вы это не сделаете, обновления безопасности не смогут быть полностью применены, новые или поддерживаемые устройства могут не функционировать и новые службы могут быть недоступны.
Пожалуйста, удостоверьтесь, что вы сохранили все ваши несохраненные документы перед перезагрузкой.
Перезапустить Позже
Перезапустить Сейчас

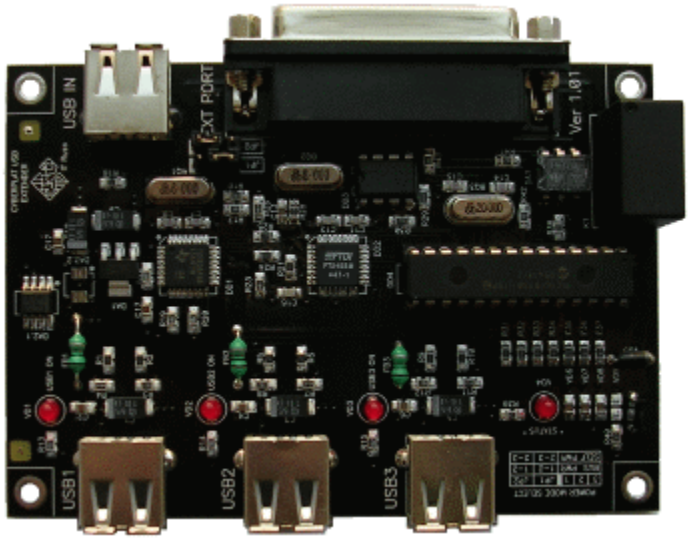
Методы отката: назад в будущее

Журнализация



Контрольные точки

- Если watchdog включен, то прикладное программное обеспечение должно не реже чем раз в минуту сообщать ему (путем вызова специальной процедуры) о своей функциональности. Если этого не происходит, например, по причине "зависания" программного обеспечения, то сторожевой таймер автоматически сгенерирует сигнал сброса для системного блока терминала, тем самым, вызвав его перезагрузку.
- Кроме того, сигнал сброса может быть инициирован самим программным обеспечением, что может быть полезно при обнаружении каких либо ошибок, устранение которых возможно путем перезагрузки системы.



Сторожевой таймер

```
new Thread(() -> {
        while (true) {
            try {

Thread.sleep(TIMEOUT_MS);
                if (!isAlive.get()) {

System.err.println("[Watchdog] Поток
завис! Перезапуск...");
                    workerThread.interrupt();
                    workerThread = new
Thread(workerThread.getRunnable());
                    workerThread.start();
                }
                isAlive.set(false); // Сброс
для следующей проверки
            } catch (InterruptedException e)
{
                e.printStackTrace();
            }
        }
    }).start();
```

```
import java.util.concurrent.atomic.AtomicBoolean;

public class WatchdogExample {
    private static final AtomicBoolean isAlive = new
AtomicBoolean(true);
    private static final int TIMEOUT_MS = 5000; // 5
секунд на реакцию

    public static void main(String[] args) {
        // Запускаем поток, который нужно мониторить
        Thread workerThread = new Thread(() -> {
            try {
                while (true) {
                    // Имитация работы
                    Thread.sleep(1000);
                    isAlive.set(true); // "Пингуем" watchdog
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });
        workerThread.start();
```

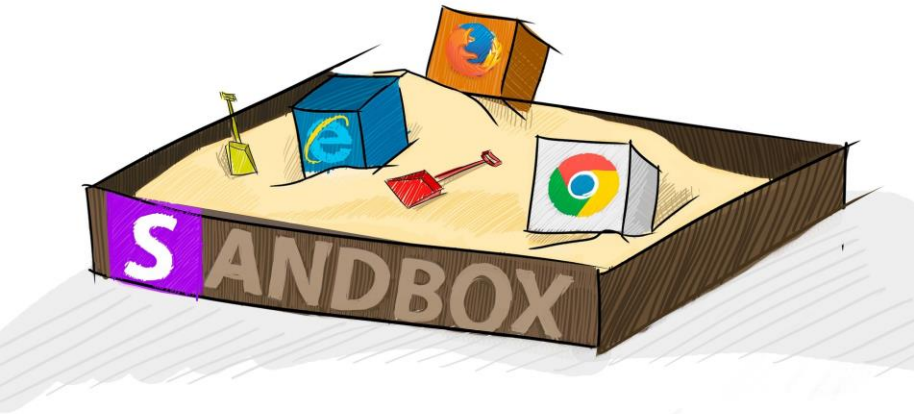
Методы *изоляции ошибок*

Изоляция ошибок — это подход к ограничению влияния сбоев в одной части системы на другие компоненты.

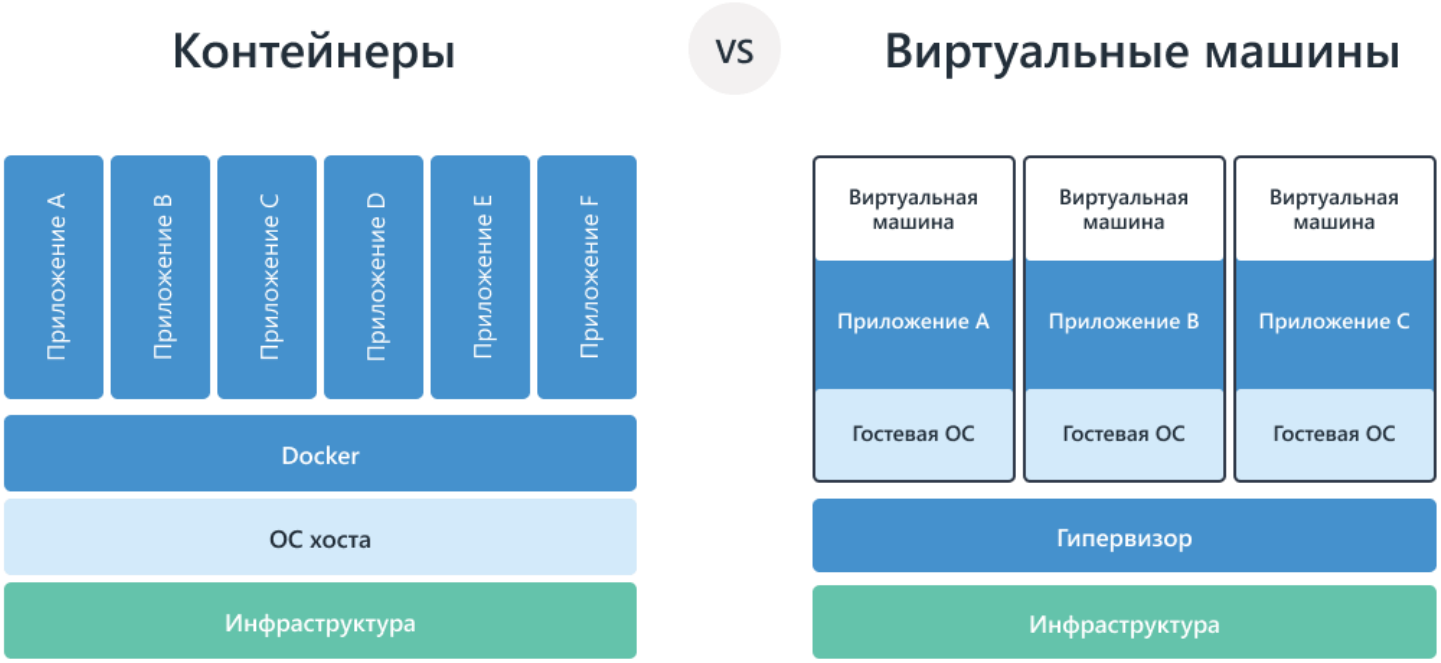
- Разделение системы на независимые сервисы
- Circuit Breaker (Автоматический выключатель), к-й При частых ошибках временно блокирует вызовы проблемного сервиса.
- Bulkhead (Разделение ресурсов), т.е. Выделение отдельных пулов потоков/памяти для разных задач.
- Изоляция через процессы/контейнеры, т.е. запуск компонентов в отдельных процессах (Docker, Kubernetes).

Методы *изоляции ошибок*

```
module myapp {
    requires java.base;          // Базовый модуль Java (подключается по умолчанию)
    exports com.myapp.api;       // Только этот пакет доступен извне
    // Пакет com.myapp.internal НЕ экспортируется — он изолирован
}
```



контейнеризация



eternalhost